

Amazon EKS Cluster Version Upgrade Guide

A step-by-step procedure for checking upgrade eligibility and upgrading an EKS control plane, add-ons, and node groups — current as of the EKS 1.36 release cycle (standard support: 1.34, 1.35, 1.36).

The core rule: one minor version at a time

EKS only allows upgrading one minor version per operation (e.g. 1.33 → 1.34, then 1.34 → 1.35, and so on). If your cluster is several versions behind, plan on multiple sequential upgrade cycles rather than one jump.

1. Check whether the upgrade is actually possible

Before assuming a target version will work, verify it with EKS's own eligibility checks rather than just attempting the upgrade.

Confirm the target version is valid

List supported Kubernetes versions and their lifecycle status, and check your cluster's current version:

```
aws eks describe-cluster-versions --cluster-versions 1.35

aws eks describe-cluster --name my-cluster --query "cluster.version"
```

versionStatus in the output tells you whether the target is STANDARD_SUPPORT, EXTENDED_SUPPORT, or UNSUPPORTED. Remember the one-minor-version-at-a-time rule below: the version may be valid in general but still unreachable in a single call from your current version.

Check EKS Upgrade Insights

This is the real answer to “can I upgrade”: EKS runs automated, recurring checks against your specific cluster for upgrade-blocking issues — deprecated or removed API usage, add-on incompatibilities, and other problems that would break on the next version.

Console: open the cluster in the EKS console → the “Upgrade insights” tab. Each flagged issue links to remediation guidance.

CLI:

```
aws eks list-insights --cluster-name my-cluster
aws eks describe-insight --cluster-name my-cluster --id <insight-id>
```

Insights refresh automatically every 24 hours, or on demand. Since March 2025, EKS has enforced this at the API level: update-cluster-version can refuse to proceed while unresolved upgrade-blocking issues exist. (AWS has toggled this enforcement at points, so check insights explicitly — don't rely on the call failing safely.) If you need to push through anyway on a non-production cluster, there's an override:

```
aws eks update-cluster-version --name my-cluster \
--kubernetes-version 1.35 --force
```

--force bypasses the upgrade-blocking readiness checks — read the insight detail first before using it on anything that matters.

Check add-on compatibility

Separately confirm your VPC CNI, CoreDNS, and kube-proxy have versions compatible with the target:

```
aws eks describe-addon-versions --addon-name vpc-cni \
--kubernetes-version 1.35
```

Putting it together — the check-before-you-upgrade sequence:

```
aws eks describe-cluster --name my-cluster --query "cluster.version"
aws eks describe-cluster-versions --cluster-versions 1.35
aws eks list-insights --cluster-name my-cluster
aws eks describe-addon-versions --addon-name vpc-cni --kubernetes-version 1.35
```

If insights come back clean and the add-ons have compatible versions available, the cluster is clear for update-cluster-version without needing --force.

2. Pre-upgrade checks

Before touching anything, work through this list:

- Read the Kubernetes/EKS release notes for every version you're passing through — each minor bump can deprecate or remove APIs (recent examples include the gitRepo volume type and Ingress-NGINX deprecations).
- Scan manifests for deprecated or removed API usage with a tool like kubent (kube-no-trouble) or pluto.
- Check that managed add-ons — VPC CNI, CoreDNS, kube-proxy, and anything else — have versions compatible with the target Kubernetes version (aws eks describe-addon-versions shows compatibility).
- Set PodDisruptionBudgets on anything critical so node rotation later doesn't take out all replicas of a service at once.
- Note that Fargate profiles, self-managed node groups, and Karpenter each have a slightly different upgrade mechanic — see Section 5.

3. Upgrade the control plane

Run the version update against the cluster itself:

```
aws eks update-cluster-version \
--name my-cluster \
--kubernetes-version 1.35
```

AWS performs this as a managed, zero-downtime, in-place replacement of the API server fleet. It typically takes 20–40 minutes. Existing workloads keep running throughout — this step only touches the control plane, not your nodes.

4. Update the add-ons

Once the control plane upgrade completes, bring the managed add-ons up to a version compatible with the new Kubernetes version:

```
aws eks update-addon --cluster-name my-cluster --addon-name vpc-cni \
--addon-version <version> --resolve-conflicts OVERWRITE

aws eks update-addon --cluster-name my-cluster --addon-name coredns \
--addon-version <version> --resolve-conflicts OVERWRITE

aws eks update-addon --cluster-name my-cluster --addon-name kube-proxy \
--addon-version <version> --resolve-conflicts OVERWRITE
```

Do this **before** upgrading nodes — an out-of-date CNI or kube-proxy running on newly upgraded nodes is a common source of networking issues.

5. Upgrade your nodes

The approach differs by node type:

Managed node groups

EKS can orchestrate this for you:

```
aws eks update-nodegroup-version \
--cluster-name my-cluster \
--nodegroup-name my-nodegroup
```

This cordons old nodes, drains pods onto new nodes running the updated AMI, then terminates the old instances. Control the rollout pace with `maxUnavailable` / `maxUnavailablePercentage` in the node group's update config.

Self-managed node groups

You own this end to end: build a new launch template with the updated AMI, roll out new instances (via ASG instance refresh or a blue/green node group swap), drain the old nodes with `kubectl drain`, then terminate them.

Fargate

Nothing to do manually — new pods scheduled after the control plane upgrade automatically land on the new Kubernetes version. Existing Fargate pods pick it up on their next restart or reschedule.

Karpenter-managed nodes

Karpenter provisions nodes based on the AMI/version referenced in your NodePool or EC2NodeClass. Update that reference and let Karpenter cycle nodes naturally, or force a drift-triggered replacement.

6. Post-upgrade validation

- Confirm cluster and node versions actually match: `kubectl get nodes` and `aws eks describe-cluster`.
- Check that everything is scheduled and healthy — no pods stuck in Pending or CrashLoopBackOff, no failed deployments.
- Run smoke tests against critical services.
- Watch logs and metrics for a while post-upgrade — some issues, like a deprecated API a workload silently depended on, only surface under real traffic.

Worth flagging before you plan the cadence

If a cluster is running a version past its 14 months of standard support, AWS charges extra per cluster-hour for versions in extended support — check where the target version sits in that lifecycle first. And always test the upgrade path on a non-production cluster if you have one: deprecated API removals are the single most common thing that breaks silently during EKS upgrades.

Sources

Review release notes for Kubernetes versions on standard support — Amazon EKS:

docs.aws.amazon.com/eks/latest/userguide/kubernetes-versions-standard.html

How to Upgrade EKS Cluster Versions — oneuptime.com/blog/post/2026-02-12-upgrade-eks-cluster-versions

Amazon EKS and Amazon EKS Distro now supports Kubernetes version 1.36 —

aws.amazon.com/about-aws/whats-new/2026/06/amazon-eks-distro-kubernetes-version-1-36

Prepare for Kubernetes version upgrades and troubleshoot misconfigurations with cluster insights —

docs.aws.amazon.com/eks/latest/userguide/cluster-insights.html

Amazon EKS now enforces upgrade insights checks as part of cluster upgrades —

aws.amazon.com/about-aws/whats-new/2025/03/amazon-eks-enforces-upgrade-insights-check-cluster-upgrades

`update-cluster-version` / `describe-cluster-versions` — AWS CLI Command Reference:

docs.aws.amazon.com/cli/latest/reference/eks/